Nemanja Vujadinovic
vujadinovic01@gmail.com

1 Question 1

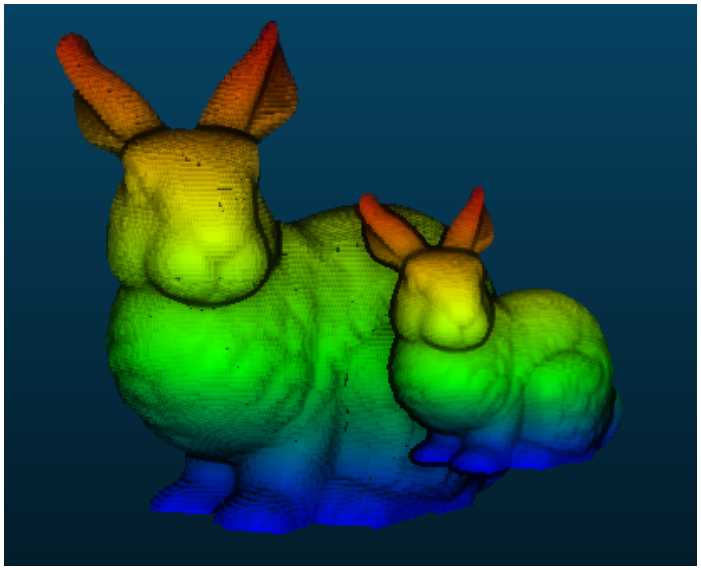


Figure 1: Original and transformed bunny.

2 Question 2

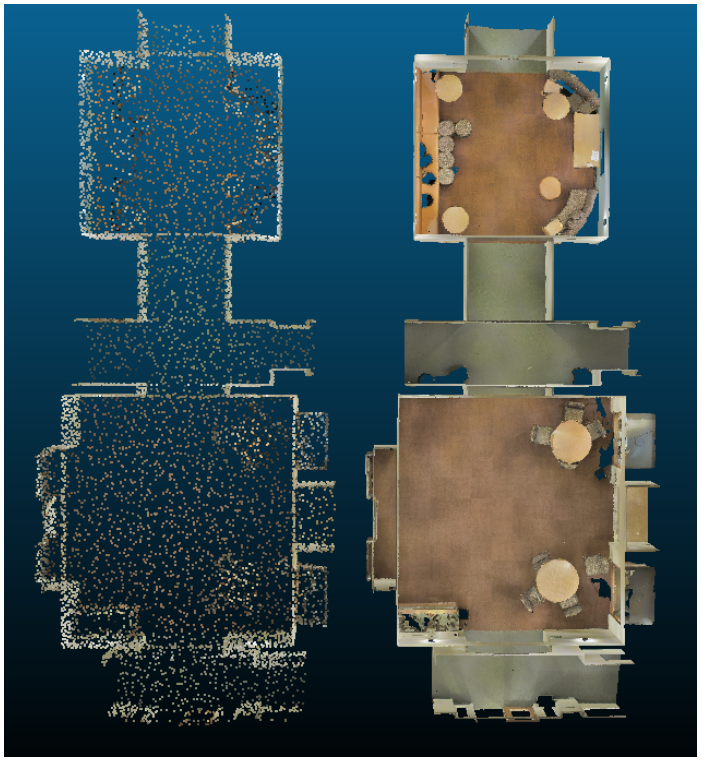


Figure 2: Decimated and original *indoor scan* point cloud (with factor $k = 300$).

3 Question 3

Observations are the following:

- Spherical neighborhoods with 10 quries and $r = 0.2$m take **0.415 seconds** to be computed.
- KNN with 10 queries and $k = 100$ take **0.592 seconds** to be computed.
- Spherical neighborhoods on whole cloud with $r = 0.2$m would take **35 hours** to be computed.
- KNN on whole cloud with $k = 100$ would take **50 hours**.

4 Question 4a

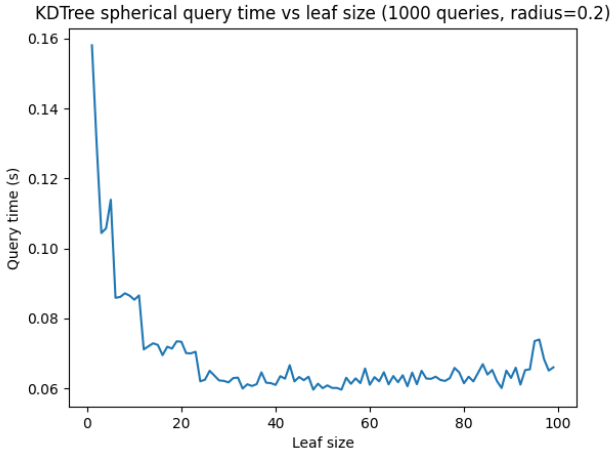


Figure 3: KD-tree spherical query time as a function of leaf size with 1000 queries and 0.2m radius.

By computing the required query time for leaf sizes in the range $\{1, 2, \ldots, 100\}$, we observe that a leaf size of **48** yields the fastest spherical neighborhood search. The behavior shown in Fig. 3 is consistent with the KD-tree documentation.[1]. For small leaf sizes, there is a small amount of brute-force work, but the KD-tree becomes deep, which increases the traversal of the tree. In contrast, larger leaf sizes produce shallower trees but require more brute-force computation. Therefore, the intermediate leaf sizes provide a trade-off between both and represent the optimal choice.

With a leaf size of 1, the KD-tree keeps splitting until each leaf contains a single point. As a result, the tree becomes very deep, which increases traversal overhead and makes this choice suboptimal.

5 Question 4b

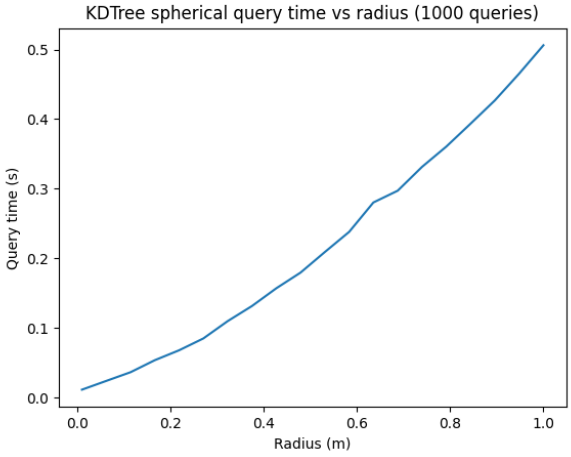


Figure 4: KD-tree spherical query time as a function of search radius with 1000 queries.

[1] `https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KDTree.html`

As shown in Fig. 4, the query time increases monotonically and non-linearly with increasing radius. For a small radius, the query time is low as only a few points fall inside the spherical neighborhood. As the radius increases, more points must be visited, and therefore, the computation time grows.

If the query search were performed on the entire point cloud with $r = 0.2$m, it would require approximately **192 seconds**.

6 Question 5

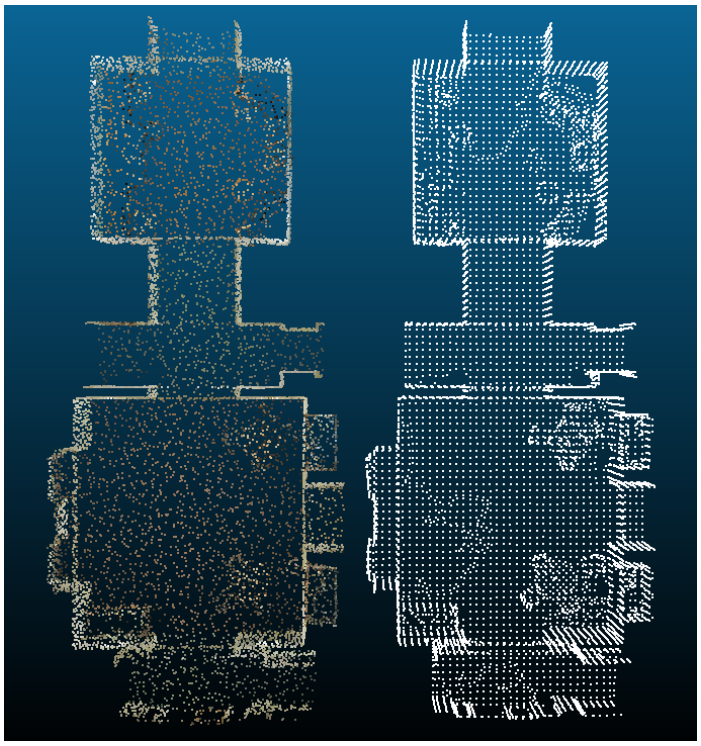


Figure 5: Decimated and grid subsampled *indoor scan* point cloud.

From a quantitative perspective, decimation subsampling requires less computation time and is easier to implement than grid subsampling. However, in qualitative terms, grid subsampling produces a more uniform and structure-preserved point cloud, with regular grid-like distribution and better geometry preservation. The points are spread more consistently and regularly across the space. In contrast, decimation subsampling produces an uneven point distribution, in which some regions are overly dense while others become sparse.